

DIGITAL IMAGE PROCESSING (DIP) SEMESTER MINI-PROJECT PROPOSAL

Project Title:

Efficient Visual Token Pruning for Vision-Language Question Answering: A Comparative Study of Saliency-Based Reduction in LLaVA

Team Details:

- **Meet Nachanekar** - Roll Number: 23108B0019
- **Arjun Kawale** - Roll Number: 23108B0020
- **Department & Class:** EXCS-B
- **Repository:** <https://github.com/Meet3724/DIP-Visual-Token-Pruning-VQA>

1. Domain & Problem Relevance

- **Domain:** Document Image Processing, Visual Information Extraction and Pattern Recognition.
- **Problem Context:** Statistical charts, plots and document graphics are dense visual representations where quantitative trends, axis scales, and structural markers are embedded in specific spatial coordinates. While humans effortlessly focus on relevant data elements (e.g., bar heights or line peaks), modern Vision-Language Models (such as LLaVA-1.5-7B) divide an input image uniformly into a static grid of 14×14 pixel patches.
- **Relevance to DIP:** In chart imagery, large portions of the image consist of uniform background whitespace, margin padding and non-informative pixels. Allocating uniform computational resources across all 576 image patches introduces computational bottlenecks in self-attention mechanisms and inflates GPU memory footprint. Integrating classical Digital Image Processing principles, specifically **Region of Interest (ROI) extraction, spatial saliency mapping and spatial domain redundancy reduction**, enables selective visual processing prior to sequence ingestion by the language model.

2. Problem Statement

To design and evaluate a query-aware visual token pruning framework that leverages digital image saliency and cross-modal semantic similarity to dynamically filter uninformative spatial image patches from chart documents, reducing visual sequence length and inference latency while preserving critical data markers required for factual question answering.

3. Specific Project Objectives

1. **Curate and Preprocess Structured Chart Datasets:** Extract and normalize benchmark chart images across diverse chart topologies (bar, line and pie plots) to analyze component distributions.
2. **Formulate Saliency-Guided Visual Filtering:** Implement a hybrid patch scoring mechanism that measures intrinsic visual feature saliency against prompt-driven cross-modal relevance.

3. **Preserve 2D Spatial Raster Topography:** Develop an indexed spatial retention algorithm that maintains 2D coordinate order among surviving visual tokens, preventing geometric disorientation during language model cross-attention.
4. **Benchmark Efficiency vs. Reasoning Degradation:** Evaluate token pruning budgets (100%, 50%, 25% and 10%) against peak VRAM utilization, GPU forward latency and answer accuracy on the ChartQA benchmark.

4. Literature Survey & Existing Approaches

- **Heuristic & Classical OCR Parsers:** Early chart processing systems (e.g., Chart-Text, Revision) used rule-based edge detection and optical character recognition. While lightweight, they fail under variable chart formatting, unusual color maps or low resolution.
- **Deep Component Detectors:** Approaches utilizing Mask R-CNN or Faster R-CNN detect discrete graphical marks (e.g., bounding boxes for bars) but require separate downstream pipelines for textual association, causing cascading detection errors.
- **Uniform Vision Transformers (ViT) & VLMs:** Architectures like CLIP ViT-L/14 divide a 336×336 image into 576 discrete patch tokens. Standard multimodal pipelines (e.g., LLaVA) feed all 576 tokens into the language model regardless of informational density. Recent literature highlights that attention sinks and background tokens consume unnecessary quadratic attention compute without contributing to reasoning fidelity.

5. Proposed Methodology

The architecture comprises four modular stages:

1. **Visual Patch Decomposition:** The input chart image is normalized to $336 \times 336 \times 3$ and decomposed into a grid of 576 distinct 14×14 pixel patches via the CLIP ViT backbone, extracting 1024-dimensional feature representations.
2. **Hybrid Saliency & Query Scoring:**

$$\text{Score}_i = \alpha \cdot S_{\text{prompt}}(i) + (1 - \alpha) \cdot S_{\text{visual}}(i)$$

where S_{prompt} computes the maximum cosine similarity between the projected 4096-D patch embedding and user question token embeddings, and S_{visual} calculates the cosine similarity against CLIP's global [CLS] token.

3. **Top-K Selection & Raster Order Preservation:** The highest-ranked patches corresponding to the target budget ratio are retained. Selected indices are strictly sorted in ascending order to preserve original top-to-bottom, left-to-right spatial context.
4. **Multimodal Sequence Fusion & Generation:** The pruned visual embeddings are concatenated with prefix instructions and suffix query tokens, forming a condensed sequence fed into the Vicuna-7B LLM for zero-shot text generation.

6. Dataset Description

- **Benchmark Source:** HuggingFaceM4/ChartQA (derived from real-world financial, governmental and academic publications).
- **Curated Test Bed:** 30 high-variance chart images cataloged in dataset/metadata.csv (comprising vertical bar charts, grouped bar charts, multi-line graphs and proportional pie charts).
- **Metadata Fields:** image_id, filename, width_px, height_px, question and ground_truth_answer.

7. Expected Outcomes & Deliverables

- A working Python implementation demonstrating up to 75% visual sequence compression (576 → 144 tokens).
- Measurable reductions in inference latency and memory requirements on commodity GPU environments (e.g., NVIDIA T4).
- Qualitative spatial heatmaps demonstrating that retained tokens cluster around critical axes, data bars, legends and query-referenced labels.

8. Development Tools & Repository Layout

- **Environment:** Python 3.10+, PyTorch, Hugging Face transformers, OpenCV, Pillow, Pandas, Matplotlib.
- **Repository Structure:**
DIP-Visual-Token-Pruning-VQA/

```
|— dataset/
|   |— images/      # 30 Curated benchmark chart images
|   |— metadata.csv # Structured ground-truth question/answer pairs
|— docs/
|   |— Task1_Project_Proposal.pdf
|— requirements.txt
|— README.md
```